

Báo cáo Phân tích Kỹ thuật: Chiến lược Tối ưu hóa Toàn diện Kiến trúc Phục vụ LLM (vLLM) cho LFM2.5-1.2B-Instruct Dưới Ràng buộc CPU Cực đoan

1. Dẫn nhập và Phân tích Không gian Bài toán (Problem Space Analysis)

Trong kỷ nguyên của các Mô hình Ngôn ngữ Lớn (LLMs), việc triển khai các hệ thống phục vụ (serving) đòi hỏi sự am hiểu sâu sắc không chỉ về kiến trúc mạng nơ-ron mà còn về sự tương tác vi mô giữa các thành phần phần cứng. Bài toán hiện tại đặt ra một thách thức cực kỳ đặc thù: triển khai và tối ưu hóa hệ thống vLLM cho mô hình LiquidAI/LFM2.5-1.2B-Instruct trên một hạ tầng có sự mất cân bằng nghiêm trọng về năng lực tính toán. Cụ thể, hệ thống được trang bị một phân vùng MiG của siêu GPU H200 với 18GB VRAM—một tài nguyên khổng lồ về băng thông bộ nhớ và năng lực xử lý dấu phẩy động—nhưng lại bị thắt cổ chai một cách cực đoan ở máy chủ vật lý với chỉ 3 nhân CPU và 8GB RAM hệ thống.

Mục tiêu tối thượng của chiến dịch tối ưu hóa này là tối đa hóa điểm số Phản hồi Yêu cầu Hiệu quả (Effective Request Score - ERS) trong quá trình đánh giá vòng trực tuyến (online round). Với đặc tả tải trọng (workload) mô phỏng môi trường sản xuất—70 hội thoại đến đồng thời theo phân phối Poisson, tổng cộng 330 yêu cầu được chấm điểm, độ dài ngữ cảnh đầu vào (context input) có thể lên tới xấp xỉ 4,000 token và độ dài đầu ra tối đa là 200 token—hệ thống sẽ phải đối mặt với áp lực duy trì đồng thời hàng chục trạng thái bộ nhớ.

1.1 Phân tích Toán học của Chỉ số ERS và Chiến lược Đánh đổi

Hệ thống chấm điểm ERS được cấu thành từ hai chỉ số thời gian thực thi cốt lõi: Thời gian đến Token Đầu tiên (Time To First Token - TTFT) và Thời gian cho mỗi Token Đầu ra (Time Per Output Token - TPOT). Công thức tổng quát được định nghĩa như sau:

$$ERS = \frac{1}{N} \sum_{i=1}^N S_{request,i} \in$$

Điểm số của từng yêu cầu $S_{request}$ được tính bằng trung bình có trọng số của hai thành phần điểm chuẩn hóa s_{tft} và s_{tpot} (với trọng số cân bằng $w = 0.5$):

$$S_{request} = w \cdot s_{tft} + (1 - w) \cdot s_{tpot}$$

Các thành phần điểm chuẩn hóa này tuân theo một hàm lũy thừa ($\gamma = 2$) dựa trên mức độ tiệm cận của TTFT và TPOT thực tế so với các ngưỡng Sàn (Floor) và Trần (Ceiling) đã được thiết lập sẵn. Sự hiện diện của hàm kẹp (clamp) giới hạn giá trị trong khoảng \$\$ trước khi áp dụng lũy thừa mang ý nghĩa vật lý rất quan trọng. Cụ thể, hàm số đánh giá TTFT:

$$s_{\{ttft\}} = \leftarrow^2$$

Và tương tự cho TPOT:

$$s_{\{tpot\}} = \leftarrow^2$$

Với các tham số cấu hình: $F_{ttft} = 10\text{ms}$, $C_{ttft} = 400\text{ms}$, $F_{tpot} = 1\text{ms}$, và

$C_{tpot} = 10\text{ms}$. Hàm lũy thừa bậc hai ($\gamma = 2$) tạo ra một đường cong phi tuyến tính, trừng phạt cực kỳ nặng nề các hệ thống có độ trễ nằm ở phần giữa của khoảng [Floor, Ceiling]. Để đạt được điểm số tối đa tiệm cận 1.0, hệ thống không thể chỉ "nhẹ nhàng vừa đủ", mà phải bút phá để đẩy TTFT xuống sát ngưỡng 10ms và TPOT xuống sát ngưỡng 1ms. Đây là những con số mang tính thách thức giới hạn vật lý của ngay cả các hệ thống suy luận biên (edge inference systems) tối tân nhất.

Thêm vào đó, cơ chế Accuracy Gate (Cổng Chất lượng) sau vòng trực tuyến mở ra một không gian đánh đổi (trade-off) mang tính chiến thuật vô cùng lớn. Việc hệ thống chấp nhận sự suy

giảm độ chính xác (Δ) so với mức tham chiếu mặc định (BF16 baseline) lên đến 0.1 (

$\Delta \leq 0.1$) mà không phải chịu bất kỳ hình phạt nào ($f(\Delta) = 1.0$) là một "giấy phép" cho phép chúng ta áp dụng các kỹ thuật lượng tử hóa cực đoan và các phương pháp giải mã suy đoán (speculative decoding) táo bạo nhất, miễn là sự suy giảm nằm trong biên độ an toàn này.

1.2 Nhận diện Nút thắt Cổ chai (Bottleneck Identification)

Một mô hình ngôn ngữ lớn cỡ 1.2 tỷ tham số như LFM2.5-1.2B-Instruct khi chạy trên GPU Hopper H200 (kiến trúc cung cấp hàng trăm teraflops cho các phép toán ma trận nhỏ) sẽ có thời gian tính toán thuần túy (model computing time) cho một lượt truyền tiến (forward pass) cực kỳ nhỏ, chỉ tính bằng vài chục microsecond.¹ Tuy nhiên, tại sao TTFT và TPOT lại chậm ngưỡng giới hạn? Câu trả lời nằm gọn trong cụm từ: **CPU Scheduling Overhead** (Chi phí điều phối của bộ xử lý trung tâm).

Với chỉ 3 nhân CPU khả dụng³, mọi tác vụ từ việc lắng nghe kết nối mạng (networking I/O), xử lý phân tích giao thức HTTP, định tuyến (routing) bằng Rust/Python, phân tích cú pháp chuỗi JSON, chạy bộ mã hóa (tokenizer), cho đến các thuật toán phức tạp của bộ lập lịch PagedAttention đều phải tranh giành chu kỳ xung nhịp (clock cycles) của 3 nhân này.⁵ Trong vLLM mặc định, toàn bộ quá trình chuẩn bị đầu vào (input preparation) và duy trì trạng thái của hàng chục yêu cầu đồng thời được thực hiện trên CPU thông qua trình thông dịch Python (Python interpreter). Sự kiện khóa toàn cục (Global Interpreter Lock - GIL) của Python kết hợp với sự khan hiếm nhân CPU vật lý dẫn đến hiện tượng GPU H200 phải chờ đợi (GPU starvation).⁶ GPU nhanh đến mức nó hoàn thành việc sinh ra một token trước khi CPU kịp nhận tín hiệu, lập lịch cho token tiếp theo và đẩy dữ liệu lên qua bus PCIe.

Sự tối ưu hóa thông thường như lượng tử hóa mô hình xuống FP8, lượng tử hóa KV Cache

xuống FP8, bật tính năng chunked prefill hay prefix caching đều là những kỹ thuật giảm tải tính toán và bộ nhớ trên GPU.¹ Chúng hoàn toàn không giải quyết được vấn đề CPU đang bị quá tải. Kể cả việc sử dụng bộ định tuyến Rust (Rust frontend) cũng chỉ tối ưu hóa được lớp mạng (network layer), trong khi bộ lõi của Engine (Engine Core) vẫn đang vùng vẫy với hàng nghìn khối bộ nhớ PagedAttention trên CPU. Báo cáo này sẽ đề xuất một hệ thống giải pháp tái cấu trúc triệt để, can thiệp sâu vào nhân hệ thống (kernel-level), kiến trúc bộ nhớ, vòng lặp điều phối của vLLM và thuật toán giải mã để "giải cứu" CPU, từ đó tối đa hóa TTFT và TPOT.

2. Phân tích Đặc tính Kiến trúc Mô hình

LFM2.5-1.2B-Instruct

Để tối ưu hóa một mô hình, ta không thể chỉ coi nó là một hộp đen (black box). Kiến trúc nội bộ của mô hình quyết định cách framework phục vụ (serving framework) tương tác với phần cứng. Dòng mô hình Liquid Foundation Models (LFM) phiên bản 2.5 mang một kiến trúc thiết kế đột phá, khác biệt hoàn toàn so với các mô hình Transformer thuần túy truyền thống như LLaMA hay Qwen.¹

2.1 Kiến trúc Lai (Hybrid Architecture) và Cấu trúc Khối

Theo các công bố đặc tả mô hình, LFM2.5-1.2B-Instruct sở hữu 1.17 tỷ tham số, được phân bổ xuyên suốt 16 lớp (layers).² Điểm làm nên sự khác biệt cốt lõi là sự kết hợp của 10 khối tích chập LIV cổng kép (double-gated LIV convolution blocks) và 6 khối Chú ý Truy vấn Nhóm (Grouped-Query-Attention - GQA blocks).² Sự phân bổ này cho thấy Liquid AI đã tối ưu hóa mô hình cho các môi trường triển khai biên (edge deployment) và thiết bị di động, nơi tài nguyên bộ nhớ cực kỳ hạn chế. Lớp tích chập LIV giúp mô hình học các biểu diễn chuỗi thời gian cục bộ với chi phí bộ nhớ đệm (KV Cache) gần như bằng không, trong khi các lớp GQA đóng vai trò thu thập ngữ cảnh toàn cục. Mô hình cũng hỗ trợ độ dài ngữ cảnh mở rộng lên tới 32,768 token, mặc dù giới hạn đánh giá của hệ thống chỉ là khoảng 4,000 token.²

2.2 Tác động của Kiến trúc Lai đối với vLLM và Hệ lụy CPU

Sự xuất hiện của các khối tích chập LIV tạo ra một thách thức rất lớn đối với kiến trúc phục vụ tiêu chuẩn của vLLM. Trong lịch sử, vLLM được thiết kế và tối ưu hóa xoay quanh kiến trúc Transformer tiêu chuẩn với cơ chế PagedAttention.¹³ Các thao tác tính toán cho Transformer đã được hợp nhất (fused) rất tốt thành các nhân CUDA khổng lồ, giảm thiểu số lượng lệnh gọi hàm từ CPU xuống GPU.

Tuy nhiên, đối với các kiến trúc lai chứa mạng tích chập chuỗi hoặc mạng trạng thái không gian (State-Space Models như Mamba hay Jamba), vLLM phải sử dụng các nhân Triton tùy chỉnh để thực thi các lớp học máy này.⁸ Vấn đề cốt lõi của Triton kernels, đặc biệt là khi triển khai trên các batch nhỏ hoặc ở giai đoạn giải mã (decode phase), là chi phí gọi hàm từ CPU (CPU launch overheads). Mỗi lần CPU gửi một lệnh yêu cầu GPU thực thi một kernel Triton, nó tốn một khoảng thời gian trễ nhất định. Với mô hình nhỏ như 1.2B, thời gian GPU tính toán một lớp tích

chập LIV có thể chỉ tốn $2\mu s$, nhưng thời gian CPU tốn để khởi chạy kernel đó (launch time) lại

lên tới $10\mu s$ đến $20\mu s$.⁸ Khi nhân lên với 10 lớp tích chập và hàng trăm token cần sinh ra, chi phí khởi chạy (launch overhead) này trở thành nguyên nhân chính khiến hệ thống không thể đưa TPOT xuống dưới 10ms, bất chấp sức mạnh của H200. Việc nhận thức được nút thắt kiến trúc này là tiền đề để chúng ta cấu hình các cơ chế Biểu đồ CUDA (CUDA Graphs) chuyên biệt nhằm khắc phục nhược điểm, một kỹ thuật sẽ được phân tích sâu ở các phần sau của báo cáo.⁸

3. Tái thiết lập Trình Thực thi Mô hình với Model Runner V2 (MRV2)

Như đã đề cập ở phần cấu hình hiện tại, các thiết lập thông thường đã đạt đến điểm bão hòa. Bước nhảy vọt đầu tiên và quan trọng nhất để phá vỡ giới hạn TPOT là chuyển đổi toàn bộ cơ chế thực thi bên trong vLLM từ kiến trúc V1 (mặc định) sang kiến trúc V2, được gọi là Model Runner V2 (MRV2).⁶ Đây là một sự chuyển đổi mô hình (paradigm shift) sâu sắc trong cách vLLM tương tác với GPU.

3.1 Hạn chế của Kiến trúc V1 (V1 Model Runner)

Trong vLLM V1, thiết kế của hệ thống phụ thuộc rất nhiều vào CPU để duy trì trạng thái của các yêu cầu đang hoạt động. Cụ thể, các cấu trúc dữ liệu mô tả lô (batch) hiện tại—bao gồm danh sách `input_ids` của các token, mảng `positions` xác định vị trí tương đối, thông tin `query_start_loc` để chỉ định điểm bắt đầu của từng chuỗi trong lô, và `seq_lens` chứa độ dài của từng chuỗi—đều được xây dựng, cấp phát và cập nhật bằng mã nguồn Python chạy trên CPU.⁶ Sau khi CPU (với 3 nhân vật lý) hoàn thành việc lắp ráp các tensor phức tạp này, nó mới kích hoạt bus PCIe để sao chép dữ liệu sang GPU.⁶ Quá trình này đòi hỏi CPU phải thực hiện hàng loạt các phép biến đổi tensor (tensor manipulation) đắt đỏ, đồng thời phải đồng bộ hóa liên tục với GPU thông qua các lệnh gọi ngầm định như `cudaDeviceSynchronize()`. Trên một máy chủ chỉ có 3 nhân CPU, việc chạy vòng lặp này cho 70 yêu cầu đồng thời (mô phỏng tải phân phối Poisson) là một gánh nặng không thể chịu đựng, dẫn đến độ trễ lập lịch (scheduling latency) tăng vọt.

3.2 Sự Đột phá của MRV2: Chuẩn bị Đầu vào Thuần GPU (GPU-Native Input Preparation)

Model Runner V2 ra đời dựa trên triết lý cốt lõi: offload (chuyển giao) tối đa mọi hoạt động có thể xuống GPU và duy trì trạng thái lưu trữ trên thiết bị (device-resident state).⁶

MRV2 tách rời hoàn toàn trạng thái yêu cầu dai dẳng (persistent request state) khỏi các tensor đầu vào của từng bước sinh token. Thay vì xây dựng lại từ đầu mảng dữ liệu ở mỗi chu kỳ xung nhịp, MRV2 phân bổ cho mỗi yêu cầu đang hoạt động một hàng tĩnh trong một bảng trạng thái (state table) có kích thước cố định nằm trực tiếp trên VRAM của GPU H200.⁶ Xuyên suốt vòng đời của yêu cầu đó, vị trí của nó trong bảng không bao giờ thay đổi, loại bỏ hoàn toàn nhu cầu phải sắp xếp lại (reorder) hay sao lưu trạng thái (như cấu trúc `CachedRequestState` mong manh trong V1).

Tại mỗi bước (step), thay vì CPU phải lắp ráp tensor đầu vào, MRV2 khởi chạy các nhân Triton siêu nhẹ để tự động thu thập và xây dựng các tensor `input_ids`, `positions`, `seq_lens`... ngay trên GPU dựa trên bảng trạng thái đã có.⁶ Sự thay đổi này mang lại những lợi ích vô song cho cấu

hình phần cứng của chúng ta:

1. **Triệt tiêu hoàn toàn CPU Overhead:** Khối lượng công việc khổng lồ của trình thông dịch Python trong việc thao tác tensor bị xóa sổ. CPU 3 nhân được giải phóng hoàn toàn khỏi nhiệm vụ chuẩn bị đầu vào, cho phép nó tập trung toàn lực vào việc xử lý mạng I/O và chạy vòng lặp sự kiện bất đồng bộ.
2. **Thiết kế Ưu tiên Bất đồng bộ (Async-First Design):** MRV2 được thiết kế với mục tiêu đồng bộ hóa bằng 0 (Zero CPU-GPU Synchronization). Bộ lập lịch (Scheduler) trên CPU có thể bắt tay vào tính toán chiến lược cho bước $N + 1$ trong khi GPU H200 vẫn đang miệt mài thực thi bước N .⁶
3. **Tối ưu Hóa Quy trình Lấy Mẫu (Sampler):** MRV2 tích hợp một trình lấy mẫu thuần Triton (Triton-native sampler) cực kỳ tối ưu. Nó sử dụng nhân lấy mẫu Gumbel-Max (Gumbel-Max Sampling Kernel), cho phép GPU thực hiện việc chọn mẫu mà không cần phải vật chất hóa (materialize) toàn bộ ma trận softmax khổng lồ trong bộ nhớ, tiết kiệm đáng kể VRAM và thời gian tính toán. Trình lấy mẫu này cũng thực hiện tính toán logprobs một cách thông minh bằng cách chỉ tính cho các ứng viên top-k đã được chọn lọc.⁶

3.3 Đánh giá Thực nghiệm và Hướng dẫn Kích hoạt

Theo các số liệu benchmark nội bộ của vLLM đối với các mô hình nhỏ (như Qwen3-0.6B) chạy trên GPU Hopper (cùng thế hệ với H200), việc sử dụng MRV2 mang lại mức tăng thông lượng lên tới 56% chỉ nhờ vào việc offload quá trình chuẩn bị đầu vào.⁶ Điều này minh chứng rõ ràng cho việc CPU bottleneck đã được giải quyết.

Để kích hoạt tính năng mang tính cách mạng này trong môi trường thi đấu, cấu hình hệ thống phải được tiêm biến môi trường trước khi khởi chạy lệnh `vllm serve`:

```
Bash
export LLM_USE_MODELRUNNERV2=1
```

Cần lưu ý rằng người dùng sẽ không cần thay đổi bất kỳ cú pháp gọi API nào. Nếu phiên bản vLLM tương thích, log hệ thống sẽ xuất ra dòng chữ `Using ModelRunnerV2`.¹⁷ Tuy nhiên, vì LFM2.5 là mô hình lai, một số nhân hỗ trợ cho tích chập tuyến tính có thể vẫn đang trong giai đoạn hoàn thiện. Trong trường hợp xấu nhất (fallback), hệ thống sẽ tự động trở về V1.¹⁹ Do đó, các bước tối ưu hóa tiếp theo vẫn cần được triển khai để tạo ra lưới an toàn đa lớp.

4. Giải mã Đầu cơ Bằng Tra cứu Tiền tố (Prompt Lookup Decoding - PLD)

Theo quy định của bài thi, trọng số ERS tập trung 50% vào TPOT (Thời gian cho mỗi Token Đầu

ra), và điểm của TPOT sẽ cực đại khi thời gian này tiệm cận 1ms.²¹ Giới hạn 1ms cho một vòng lặp mạng nơ-ron sâu (dù nhỏ gọn như 1.2B) cộng thêm chi phí truyền tải qua lại giữa máy khách và máy chủ là một thách thức cực đại. Giải pháp thông thường cho bài toán này là Giải mã Đầu cơ (Speculative Decoding). Tuy nhiên, các kỹ thuật giải mã đầu cơ truyền thống đòi hỏi việc duy trì một "mô hình nháp" (draft model) nhỏ hơn nhiều lần để dự đoán token trước khi đưa vào mô hình chính (target model) để kiểm chứng.²³ Việc này là hoàn toàn bất khả thi trong điều kiện bộ nhớ RAM của hệ thống (system RAM) bị giới hạn ở 8GB và CPU chỉ có 3 nhân, bởi việc vận hành song song hai luồng mô hình sẽ dẫn đến tình trạng cạn kiệt tài nguyên tức thì.

Giải pháp xuất sắc nhất, khai thác triệt để đặc điểm của tải trọng bài thi (multi-turn conversation, context lên tới 4000 token, output nhỏ) mà không tiêu tốn thêm bất kỳ tài nguyên mô hình nháp nào, chính là **Prompt Lookup Decoding (PLD)**.²⁴

4.1 Cơ chế Cốt lõi của PLD: Tận dụng Chuỗi N-gram

Prompt Lookup Decoding hoạt động dựa trên một hiện tượng ngôn ngữ học rất phổ biến trong các ứng dụng AI tạo sinh, đặc biệt là trong các nhiệm vụ sinh mã lập trình (code generation), xử lý định dạng JSON, phân tích tài liệu và hội thoại theo khuôn mẫu: sự lặp lại của các chuỗi từ vựng.²³

Thay vì dùng một mạng nơ-ron khác để dự đoán, PLD coi chính phần ngữ cảnh (context/prompt) đã có sẵn trong bộ nhớ đệm KV (KV cache) của mô hình làm cơ sở dữ liệu để nội suy.²⁴ Thuật toán hoạt động theo các bước sau:

1. **Theo dõi độ trùng khớp (N-gram Matching):** Khi mô hình chính sinh ra một tập hợp các

token mới (ví dụ k token cuối cùng), PLD sẽ thực hiện thuật toán tìm kiếm chuỗi (string matching algorithm) để quét ngược toàn bộ lịch sử ngữ cảnh đầu vào. Quá trình tìm kiếm

này tìm xem có đoạn nào trong ngữ cảnh cũ khớp hoàn toàn với chuỗi k -gram vừa sinh ra hay không.²²

2. **Trích xuất chuỗi dự đoán (Speculative Draft):** Nếu tìm thấy một sự trùng khớp, thuật toán sẽ giả định rằng các token tiếp theo mô hình sắp sinh ra cũng sẽ tuân theo mẫu (pattern) đã xuất hiện trong ngữ cảnh. Hệ thống lập tức trích xuất một số lượng token nhất định tiếp theo từ ngữ cảnh cũ để làm chuỗi token dự đoán (draft tokens).
3. **Kiểm chứng Song song (Parallel Verification):** Thay vì sinh từng token một (mỗi token tốn $1 \times$ thời gian truyền tiến), vLLM sẽ nhóm toàn bộ các token dự đoán này lại và đưa vào mô hình chính trong một lượt tính toán ma trận (single forward pass) duy nhất. GPU sẽ tính toán xác suất của toàn bộ chuỗi để kiểm chứng xem chuỗi dự đoán có đúng hay không.
4. **Tăng tốc độ (Speedup):** Nếu chuỗi dự đoán khớp, hệ thống đã sinh ra được ví dụ 5 token trong cùng một khoảng thời gian vốn chỉ đủ để sinh 1 token. Nếu không khớp, hệ thống đơn giản là loại bỏ phần sai và tiếp tục sinh từ điểm sai đó. Vì GPU H200 có năng lực tính toán song song quá dư thừa đối với batch size nhỏ, việc kiểm chứng 1 token hay 5 token trong một lượt forward pass gần như tốn cùng một lượng thời gian.

Kết quả là, thời gian trung bình để sinh ra một token (TPOT) giảm đi một cách phi mã (có thể

giảm từ 3 đến 5 lần), giúp TPOT vượt qua được ranh giới 1ms.

4.2 Cấu hình Tham số Tối ưu cho Bài toán Cụ thể

Việc thiết lập PLD cần được tính toán dựa trên không gian tác vụ của vòng thi. Các tham số cần thiết lập bổ sung vào cấu hình khởi chạy vllm serve bao gồm:

Tham số	Giá trị Đề xuất	Lý do và Cơ chế Toán học
--speculative-model	''	Kích hoạt bộ giải mã suy đoán dựa trên tra cứu tiên tổ thay vì yêu cầu tải trọng mô hình nháp. ²² Lệnh này tương thích nguyên bản với vLLM.
--num-speculative-tokens	5	Xác định số lượng token tối đa mà thuật toán N-gram được phép trích xuất và đưa vào kiểm chứng ở mỗi bước. ²² Giới hạn output của bài thi là 200 token. Nếu đặt số này quá lớn, tỷ lệ từ chối (rejection rate) sẽ tăng cao do xác suất chuỗi dài trùng khớp hoàn toàn là rất thấp, gây lãng phí bộ nhớ GPU. Mức 5 là điểm giao thoa tối ưu (sweet spot) được chứng minh thực nghiệm cho các mô hình ngôn ngữ quy mô nhỏ. ²²
--ngram-prompt-lookup-max	3	Độ dài của cửa sổ trượt (sliding window) dùng để định nghĩa mẫu n-gram. Đặt bằng 3 nghĩa là hệ thống sẽ dùng 3 token gần nhất vừa sinh ra làm chìa khóa (key) để tìm kiếm trong ngữ cảnh cũ. Các giá trị lớn hơn sẽ làm tăng độ chính xác của dự đoán nhưng giảm số cơ

		hội tìm thấy (hit rate). ²²
--	--	--

Giải pháp này hoàn toàn tuân thủ các quy tắc chống gian lận (Anti-Cheating) của Ban Tổ chức vì nó không sử dụng cơ chế can thiệp đường dẫn kép (dual-path), không viết cứng kết quả (hardcode) và không gọi ra bên ngoài. Nó là một tối ưu hóa toán học thuần túy dựa trên bản chất thống kê của thông tin đầu vào. Bằng cách giảm TPOT xuống mức vi mô (microsecond

scale), thành phần điểm s_{tpot} trong công thức ERS sẽ đạt mức 1.0 tuyệt đối.

5. Cấp phát Bộ nhớ Hệ thống và Xóa bỏ Tranh chấp Khóa (Lock Contention)

Khi quan sát ở mức độ hệ điều hành (Ubuntu 24.04), vLLM là một tiến trình đa luồng cực kỳ phức tạp. Hệ thống bao gồm máy chủ HTTP do Rust (tokio) hoặc Python (asyncio) quản lý, bộ lập lịch PagedAttention, bộ theo dõi số liệu và các trình bao bọc (wrappers) để tương tác với driver GPU.²⁶

Bài toán đặt ra: 70 yêu cầu theo phân phối Poisson đến liên tục. Mỗi yêu cầu mang theo chuỗi văn bản dài hàng nghìn ký tự. Hệ thống phải liên tục khởi tạo, cập nhật và hủy bỏ hàng vạn đối tượng Python, chuỗi ký tự, và danh sách các tensor trên bộ nhớ RAM hệ thống (8GB).

5.1 Vấn đề của Trình Cấp phát Bộ nhớ Mặc định (glibc malloc)

Theo mặc định, các hệ thống Linux như Ubuntu sử dụng glibc malloc làm trình quản lý bộ nhớ động. Khi các luồng khác nhau của vLLM cùng đồng thời yêu cầu hệ điều hành cấp phát một đoạn RAM nhỏ để chứa dữ liệu mới, glibc phải sử dụng các cơ chế khóa đồng bộ (mutex locks) để bảo vệ vùng nhớ heap chung (global heap), ngăn chặn tình trạng ghi đè dữ liệu (data corruption).

Trên cấu hình đa nhân nhưng số lượng ít (3 nhân), khi hàng nghìn yêu cầu cấp phát bộ nhớ được bắn ra mỗi giây, các luồng sẽ liên tục rơi vào trạng thái chờ đợi lẫn nhau (Lock Contention).²⁸

Hiện tượng này làm CPU phải tốn chu kỳ xung nhịp chỉ để quản lý các khóa thay vì thực hiện tính toán. Nó là tác nhân chính gây ra các gai độ trễ ngẫu nhiên (latency spikes), làm tăng đột biến chỉ số TTFT ở phân vị thứ 95 (p95 TTFT - một tiêu chí tie-break cực kỳ quan trọng trong vòng chung kết).³⁰

5.2 Triển khai TCMalloc (Thread-Caching Malloc)

TCMalloc (được phát triển bởi Google) ra đời để giải quyết triệt để nút thắt đa luồng này.²⁸ TCMalloc áp dụng một kiến trúc bộ nhớ đệm đa tầng (multi-level cache structure). Thay vì mọi luồng đều phải tranh giành một vùng nhớ heap dùng chung, TCMalloc phân bổ cho *mỗi luồng đang chạy* một vùng nhớ đệm cục bộ (thread-local cache) riêng biệt. Khi một luồng của vLLM (ví dụ luồng lập lịch token) cần cấp phát đối tượng bộ nhớ, nó sẽ lấy trực tiếp từ vùng đệm cục bộ của chính nó mà không cần kiểm tra khóa toàn cục. Điều này khiến cho các hoạt động khởi tạo và hủy đối tượng (vốn xảy ra hàng triệu lần trong vòng đời phục vụ LLM) đạt tốc độ gần như tức thời.

Một giải pháp thay thế là Jemalloc.²⁸ Tuy nhiên, theo các nghiên cứu thực nghiệm, Jemalloc

phù hợp hơn khi các luồng là tĩnh (static pools). Với đặc thù của Rust tokio frontend thường xuyên khởi tạo và hủy các luồng công nhân (worker threads) để xử lý các kết nối HTTP ngắn hạn, TCMalloc mang lại hiệu suất tổng thể ưu việt hơn.²⁹

Tích hợp vào quá trình đóng gói (Dockerization): Do vòng thi yêu cầu thí sinh tự đóng gói Docker Image, việc tích hợp TCMalloc không cần can thiệp mã nguồn vLLM, chỉ cần sửa đổi cấu trúc hệ điều hành:

Dockerfile

```
# Cài đặt gói thư viện quản lý bộ nhớ
```

```
RUN apt-get update && apt-get install -y libtcmalloc-minimal4
```

```
# Ghi đè trình cấp phát bộ nhớ mặc định của Linux thông qua LD_PRELOAD
```

```
ENV
```

4

Chỉ với 2 dòng lệnh bổ sung này, hiệu suất xử lý I/O và tính cục bộ của bộ nhớ (cache locality) của CPU sẽ được cải thiện đáng kể, làm phẳng hoàn toàn đường cong độ trễ của hệ thống.³¹

6. Tối ưu Không gian Lập lịch PagedAttention và Ngân sách Token (Token Budgeting)

Một trong những thuật toán cốt lõi làm nên sức mạnh của vLLM là PagedAttention, giúp giảm sự phân mảnh bộ nhớ (memory fragmentation) cho bộ nhớ đệm KV.⁸ Tuy nhiên, trong điều kiện CPU yếu, PagedAttention có thể trở thành một con dao hai lưỡi nếu tham số kích thước khối (block size) không được điều chỉnh.

6.1 Sự Lạm phát Khối Bộ Nhớ (Block Inflation Overhead)

Theo thiết lập mặc định, vLLM sử dụng `--block-size 16`, nghĩa là mỗi khối bộ nhớ liền kề trong VRAM sẽ chứa dữ liệu Key-Value của đúng 16 token.³³ Hãy cùng thực hiện một phép tính toán học hệ thống dựa trên yêu cầu của vòng thi:

- Mỗi ngữ cảnh đầu vào tối đa: $\approx 4,000$ token.
- Số lượng khối cần thiết cho một yêu cầu: $\frac{4000}{16} = 250$ khối.
- Với phân phối Poisson 70 yêu cầu diễn ra đồng thời: $70 \times 250 = 17,500$ khối.

Ở mỗi bước sinh (decoding step), bộ lập lịch PagedAttention (chạy bằng Python trên CPU) phải lặn qua cấu trúc dữ liệu mô tả ánh xạ của toàn bộ 17,500 khối này.³⁴ Nó phải quyết định xem khối nào còn trống, khối nào cần hoán đổi ra RAM hệ thống (swap out), khối nào cần nạp lại (swap in). Một vòng lặp for hoặc while quét qua 17,500 đối tượng phức tạp trong một môi trường CPU bị ép xung và giới hạn bộ nhớ đệm L3 sẽ tiêu thụ vài phần nghìn giây. Con số này

phá hỏng hoàn toàn mục tiêu TPOT < 1ms.

6.2 Giải pháp Đánh đổi: Mở rộng Kích thước Khối (Block Size Enlargement)

Chiến lược kỹ thuật đột phá ở đây là khai thác sự dư thừa của GPU H200 để bù đắp cho sự thiếu hụt của CPU. GPU H200 được cấp có tới 18GB VRAM. Mô hình LFM2.5-1.2B-Instruct khi tải ở định dạng lượng tử hóa FP8 chỉ chiếm không quá 1.5GB đến 2GB không gian VRAM.¹ Toàn bộ phần dung lượng khổng lồ còn lại (hơn 15GB) là hoàn toàn dư thừa đối với một tải trọng 70 yêu cầu.

Ta sẽ tăng cấu hình `--block-size` lên một giá trị cực đoan hơn: 128 (mặc định trên một số hệ thống Neuron/HPU nhưng có thể cưỡng ép trên CUDA) hoặc giá trị giới hạn cấu trúc hệ thống.³³

- Với `block_size = 128`, số lượng khối cho mỗi yêu cầu giảm xuống còn $\frac{4000}{128} \approx 31$ khối.
- Tổng khối cho 70 yêu cầu giảm xuống chỉ còn khoảng 2,170 khối.

Việc bộ lập lịch trên CPU quản lý 2,170 khối thay vì 17,500 khối sẽ nhanh hơn theo cấp số nhân (độ phức tạp thời gian giảm xấp xỉ 8 lần). Sự đánh đổi ở đây là hiện tượng phân mảnh nội vi (internal fragmentation): nếu một đoạn văn bản kết thúc lè ở token thứ 10, vLLM vẫn phải cấp phát toàn bộ khối 128 token, làm lãng phí 118 token không gian VRAM.¹³ Nhưng như đã phân tích, 18GB VRAM của H200 dư sức hấp thụ sự hao phí phân mảnh này. Đổi lại, chu kỳ CPU được bảo toàn trọn vẹn để đáp ứng TPOT.

6.3 Phân mảnh Tiền tải (Chunked Prefill) và Cân bằng Ngân sách (Budget Balancing)

Hiện tại giải pháp đã sử dụng cờ `--enable-chunked-prefill`, một cơ chế rất tốt để ngăn hiện tượng chặn đầu hàng (head-of-line blocking) khi các đoạn văn bản dài 4000 token ập đến làm đình trệ các đoạn ngắn đang giải mã. Tuy nhiên, nếu cờ `--max-num-batched-tokens` chưa được tính toán kỹ, GPU H200 sẽ không được phát huy tối đa công suất.

Khi cắt nhỏ chuỗi, kích thước khối tải tiền (prefill chunk) phải đủ lớn để bão hòa (saturate) các khối Tensor (Tensor Cores) của kiến trúc Hopper. Cấu hình tối ưu được khuyến nghị:

- `--max-num-batched-tokens 4096` hoặc 8192.¹⁵
- `--max-num-seqs 128`. (Dư giả cho 70 req đồng thời, đảm bảo PagedAttention không giới hạn số chuỗi hoạt động một cách giả tạo).¹⁵

Với thiết lập này, một yêu cầu mới 4000 token có thể được xử lý gọn gàng trong 1 hoặc 2 bước forward pass, trong khi vẫn lồng ghép được cùng các token giải mã của các yêu cầu đang hiện hành, giữ cho biểu đồ ERS tăng trưởng tuyến tính mà không có điểm gãy.³⁷

7. Khắc phục CPU Launch Overhead với Biểu đồ Tính toán Toàn diện (CUDA Graphs FULL_AND_PIECEWISE)

Như đã đề cập ở Mục 2, kiến trúc chứa khối tích chập LIV của LFM2.5 tạo ra các hạt nhân phi-biểu diễn (non-transformer kernels) bằng Triton, dẫn đến độ trễ khởi chạy (launch

overhead) rất lớn từ phía CPU. Để giải quyết vấn đề CPU phải ra lệnh cho GPU ở từng microsecond, vLLM sử dụng công nghệ Biểu đồ CUDA (CUDA Graphs).⁸

7.1 Cơ chế và Hạn chế của CUDA Graphs Mặc định

Về cơ bản, CUDA Graphs là kỹ thuật cho phép CPU "chụp" (capture) lại toàn bộ chuỗi các lệnh thực thi cấu trúc lõi của mạng nơ-ron thành một biểu đồ hình cây tĩnh. Sau khi được chụp, CPU không cần phải gọi từng lệnh đơn lẻ nữa, mà chỉ cần bắn một lệnh phát lại (replay) duy nhất. GPU sẽ tự động duyệt qua biểu đồ tĩnh này. Điều này triệt tiêu hoàn toàn CPU launch overhead. Tuy nhiên, với các mô hình lai (hybrid models), hình dạng của các tensor (tensor shapes) ở các khối tích chập có tính biến động dựa trên độ dài của chuỗi. Trong vLLM V1 trước đây, biểu đồ CUDA bị hạn chế hoặc không thể hoạt động ổn định trên các mô hình lai, buộc hệ thống phải quay lại chế độ thực thi hăng hái (Eager execution), tái hiện lại bóng ma độ trễ CPU.⁸

7.2 Chiến lược Tối ưu với FULL_AND_PIECEWISE

Các phiên bản tối ưu hóa gần đây của vLLM đã giới thiệu một kỹ thuật chuyên biệt cho các kiến trúc lai (Mamba, Jamba, và các biến thể chập) mang tên FULL_AND_PIECEWISE CUDA Graphs.⁸ Kỹ thuật này cho phép vLLM xây dựng các biểu đồ đầy đủ (full graphs) cho các lô chỉ chứa yêu cầu giải mã (decode-only batches) và biểu đồ phân mảnh (piecewise graphs) cho các lô hỗn hợp (có cả tải tiền và giải mã).⁸

Sự cải thiện TPOT nhờ việc kích hoạt và tinh chỉnh CUDA graph trên các mô hình lai là vô cùng lớn. Các thử nghiệm trên mô hình lai (ví dụ: IBM granite-h-tiny) cho thấy thông lượng tăng đến 91% ở mức tải độ trễ thấp.⁸ Để đảm bảo tính năng này hoạt động toàn diện và bao phủ toàn bộ giới hạn dữ liệu của bài thi (4000 token), ta phải thiết lập:

- Đảm bảo cờ `--enforce-eager` **KHÔNG** được bật (mặc định là tắt, nhưng nhiều người dùng hay bật để gỡ lỗi).¹⁵
- Cấu hình `--max-seq-len-to-capture 4096` (hoặc giá trị ngưỡng tối đa của hệ thống benchmark).¹⁵ Khi cấu hình này được thiết lập, vLLM sẽ tốn một khoảng thời gian đầu (warm-up phase) khi khởi động bộ chứa (container) để liên tục thay đổi kích thước batch và capture toàn bộ các trường hợp biểu đồ có thể xảy ra. Trong thời gian chấm điểm thực sự (online evaluation round), đối với bất kỳ batch size nào nằm trong giới hạn đã chụp, CPU sẽ không phải tốn chu kỳ điều phối hạt nhân nào nữa, GPU sẽ chạy với vận tốc cực đại.

8. Can thiệp Hệ điều hành: Ép Luồng (CPU Affinity) và Điều phối Không gian Đa Luồng

Với kiến trúc lõi chỉ 3 nhân CPU, việc để cho hệ điều hành Ubuntu tự do quản lý việc gán luồng (thread assignment) là một thảm họa về hiệu năng. Lập lịch mặc định của hạt nhân Linux (Linux Completely Fair Scheduler - CFS) có xu hướng di chuyển các tiến trình qua lại giữa các nhân (thread migration) nhằm cân bằng tải.⁴⁰ Hệ quả là bộ nhớ đệm L1 và L2 của nhân CPU liên tục bị xóa (cache miss), buộc CPU phải nạp lại dữ liệu từ RAM chậm chạp.

8.1 Khóa Luồng Phần cứng (Thread Pinning / CPU Affinity)

Để giải quyết bài toán này, ta sử dụng công cụ cấp hệ thống taskset để tạo ra một "tấm khiên" (shield), trói buộc toàn bộ hệ sinh thái tiến trình của vLLM vào các nhân vật lý nhất định.⁴⁰ Lệnh khởi động trong tập tin docker-compose.yml hoặc script entrypoint cần được sửa đổi:

```
Bash
```

Lệnh này ra chỉ thị nghiêm ngặt cho hạt nhân Linux không được phép điều hướng tiến trình vLLM ra khỏi dải nhân từ 0 đến 2 (tổng cộng 3 nhân). Việc giữ tiến trình cố định giúp bảo toàn dữ liệu PagedAttention table trên bộ nhớ đệm tốc độ cao của CPU (L2/L3 cache), đặc biệt khi kết hợp với chiến lược quản lý của TCMalloc.⁴⁰ Đây cũng là ứng dụng thực tế hóa của các cấu trúc lập lịch Topology cây (TopoTree/Sandwich scheduling) nhằm tăng cường độ cục bộ không gian (spatial locality) của phần cứng.³

8.2 Phân bổ Không gian Luồng (Thread Space Allocation)

Người dùng đang sử dụng cấu hình: VLLM_USE_RUST_FRONTEND=1 Và thiết lập TOKIO_WORKER_THREADS=2.

Cấu hình Rust frontend là một nước đi chính xác để thay thế máy chủ Python FastAPI chậm chạp. Tokio là một runtime bất đồng bộ của Rust, cực kỳ hiệu quả trong xử lý kết nối HTTP.²⁷ Tuy nhiên, việc dành hẳn 2 luồng (tương đương gần 2/3 tổng tài nguyên nhân CPU khả dụng) chỉ để xử lý giao thức mạng là một sự lãng phí quá mức và cực kỳ nguy hiểm. Lý do: Tải trọng của hệ thống là phân phối Poisson với 70 yêu cầu. Rust frontend chỉ làm nhiệm vụ parse JSON HTTP request và đẩy nó vào hàng đợi. Nó tốn rất ít thời gian. Trong khi đó, hạt nhân Python điều phối GPU (Engine Core, Scheduler) lại phải làm những công việc phức tạp nhất, và nếu nó chỉ còn lại 1 nhân CPU để xử lý, toàn bộ hệ thống sẽ bị thắt cổ chai tại khâu lập lịch.

Đề xuất tinh chỉnh: Nên hạ cấp TOKIO_WORKER_THREADS=1. Việc này giải phóng 1 nhân CPU quý giá trả lại cho vòng lặp sự kiện bất đồng bộ chính (AsyncLLMEngine) và các quá trình xử lý nền như GPU callback.²⁶

Các cấu hình giới hạn thư viện toán học CPU đã thiết lập sẵn là hoàn toàn chuẩn xác và phải được duy trì: OMP_NUM_THREADS=1, MKL_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1.⁴² LFM2.5 tính toán hầu hết ma trận trên CUDA, việc để các thư viện BLAS sinh ra hàng chục luồng nhàn rỗi (idle threads) chỉ làm nhiễu loạn thêm bộ lập lịch của Linux, do đó việc cô lập chúng ở mức 1 luồng là cấu hình bắt buộc trong các hệ thống ràng buộc CPU.

9. Triệt tiêu Chi phí API Khung ứng dụng và Tối ưu hóa Chuỗi Ký tự (Framework Overhead & Template

Optimization)

Sau khi dọn dẹp các nút thắt về bộ nhớ, kiến trúc và hệ điều hành, rào cản cuối cùng ngăn cản TTFT đạt ngưỡng tối thiểu nằm ở các dòng lệnh Python xử lý vòng lặp API và định dạng đầu vào.

9.1 Hệ thống Viết Nhật ký (Logging Suppression)

Khung ứng dụng vLLM được thiết kế cho môi trường nghiên cứu và máy chủ cục bộ, nên hệ thống thu thập số liệu (telemetry) và ghi log của nó vô cùng nặng nề. Ở mỗi yêu cầu hoặc mỗi bước tính toán, vLLM sử dụng Python để định dạng các chuỗi trạng thái, gọi các hàm theo dõi bộ đệm, mở hệ thống tập tin (file descriptors) để ghi dữ liệu ra bộ gỡ lỗi (console stdout). Hoạt động I/O (Nhập/Xuất) này gây tắc nghẽn luồng xử lý chính.

Cần phải áp đặt lệnh "kỷ luật sắt" đối với hệ thống ghi log bằng cách cung cấp cờ khởi động:

- `--disable-log-requests` (Khóa log mọi giao dịch HTTP).⁴⁴
- `--disable-log-stats` (Khóa log theo dõi GPU utilization định kỳ).²⁷ Đồng thời, đẩy biến môi trường Python lên mức chỉ báo lỗi nghiêm trọng: `VLLM_LOGGING_LEVEL=ERROR` hoặc `CRITICAL`. Chỉ riêng thay đổi nhỏ này đã giúp cứu lại hàng mili-giây cho mỗi lượt tính TTFT trên cấu hình CPU yếu.

9.2 Vượt qua Nút thắt Phân tích Cú pháp Jinja2 (Bypass Jinja2 Chat Template Overhead)

Mô hình LFM2.5 sử dụng một cấu trúc mẫu hội thoại (chat template) nội bộ có hình thái tương tự ChatML (`<|im_start|>user...`).¹⁰ Khi hệ thống chấm thi gọi giao thức API tương thích OpenAI (OpenAI-compatible `/v1/chat/completions`), đầu vào sẽ là một mảng JSON các đối tượng tin nhắn (message objects).⁴⁷

Trong quy trình chuẩn, vLLM phải sử dụng bộ engine nội suy Jinja2 của Python để lặp qua danh sách JSON này, nối chuỗi, và áp dụng các quy tắc kiểm tra lỗi trước khi chuyển nó cho Tokenizer.⁴⁸ Quá trình xử lý chuỗi động (dynamic string processing) bằng Python rất tốn kém tài nguyên CPU. Đặc biệt với các đoạn hội thoại đa lượt (multi-turn) rất dài, vòng lặp Jinja2 sẽ làm tăng độ trễ một cách vô nghĩa.

Kỹ thuật Tối ưu: Mặc dù cấu hình hiện tại đang sử dụng `VLLM_USE_FASTTOKENS=1` (khai thác lỗi Tokenizer viết bằng Rust⁴⁹), bước áp dụng chat template bằng Jinja2 vẫn xảy ra trước đó. Để vượt qua độ trễ này một cách hợp lệ mà không vi phạm nguyên tắc chống gian lận (không sửa weights hay mã nguồn Tokenizer⁴⁸), ta có thể tạo ra một tập tin `chat_template.jinja` tối giản tuyệt đối, tước bỏ tất cả các biểu thức logic `if/else` kiểm tra định dạng phức tạp trong mẫu mặc định của HuggingFace, chỉ giữ lại logic nối chuỗi cơ bản nhất.⁵⁰ Tập tin này được đẩy vào Docker container và khởi chạy cùng với cờ `--chat-template /path/to/template.jinja`.⁴⁷

10. Bảng Cấu trúc Đề xuất và Lộ trình Triển khai Tổng thể (Implementation Roadmap)

Dựa trên các phân tích chuyên sâu về hệ thống phần cứng, kiến trúc vi mô của mô hình và rào cản luồng xử lý, lộ trình tối ưu hóa sau đây tích hợp tất cả các thành phần kỹ thuật vào một tổng

thể thống nhất. Việc kết hợp đồng thời các giải pháp sẽ tạo ra hiệu ứng cộng hưởng (synergy), giải quyết dứt điểm nút thắt cổ chai ở cả ba tầng: OS (TCMalloc, Taskset), Framework (MRV2, PLD, Block Size), và API (Logging, Tokio).

10.1 Bảng Phân bổ Tham số Cấu hình Tối ưu

Dưới đây là bảng tổng hợp các chỉ thị quan trọng nhất để xây dựng cấu hình hệ thống, ánh xạ trực tiếp đến các nguyên nhân gây nghẽn:

Hạng mục Tối ưu	Lệnh / Tham số Biến Môi trường	Cơ sở Khoa học và Tác động
Trình Cấp phát Bộ nhớ	LD_PRELOAD=...libtcmalloc..	Triệt tiêu tranh chấp khóa toàn cục (Global lock contention) trên 3 nhân CPU, làm phẳng độ trễ p95 TTFT.
Phân bổ Luồng Lập lịch	TOKIO_WORKER_THREADS=1	Trả lại 1 nhân vật lý cho vòng lặp AsyncLLMEngine thay vì lãng phí cho mạng HTTP.
Quản lý CPU Affinity	Lệnh bọc taskset -c 0-2	Khóa tiến trình vào dải nhân cố định, tối đa hóa tỷ lệ hit bộ nhớ đệm L1/L2 của CPU.
Model Runner V2	VLLM_USE_V2_MODEL_RUNNER=1	Chuyển đổi toàn bộ quá trình lắp ráp tensor chuẩn bị đầu vào sang VRAM của GPU, giải thoát CPU hoàn toàn khỏi các phép toán Python.
Block Size PagedAttention	--block-size 128	Giảm số lượng khối bộ nhớ phải quản lý từ 17,500 xuống còn $\approx 2,000$. Đánh đổi VRAM lấy thời gian vòng lặp của CPU.
CUDA Graphs Hybrid	--max-seq-len-to-capture 4096	Kích hoạt FULL_AND_PIECEWISE

		CUDA graphs cho các khối LIV convolution, loại bỏ độ trễ gọi kernel (launch overhead).
Giải mã N-Gram (PLD)	<code>--speculative-model</code> <code>--num-speculative-tokens 5</code> <code>--ngram-prompt-lookup-max 3</code>	Thuật toán đoán trước token dựa trên nội suy ngữ cảnh, nhân bản số token đầu ra trong một lượt tính toán mà không cần mô hình nháp, ép TPOT xuống dưới 1ms.
Triệt tiêu I/O Log	<code>--disable-log-requests</code> <code>--disable-log-stats</code>	Vô hiệu hóa quá trình xử lý chuỗi và ghi đĩa không cần thiết, tiết kiệm chu kỳ CPU.

10.2 Kết luận Khái quát

Việc đẩy một hệ thống AI serving đến giới hạn tối đa không chỉ là câu chuyện của việc lựa chọn tham số framework, mà là nghệ thuật cân bằng giữa cấu trúc mạng nơ-ron (hybrid LIV/GQA của LFM2.5) và tài nguyên hệ thống vật lý (Sự bất đối xứng giữa H200 và 3-core CPU). Thông qua việc đánh đổi một cách có chủ đích lượng VRAM dư thừa (chấp nhận phân mảnh nội vi bằng cách tăng block size lên 128), chúng ta mua lại được tài nguyên tính toán vô giá từ bộ lập lịch CPU. Bằng cách kích hoạt Model Runner V2 và CUDA Graphs, chúng ta chuyển giao toàn bộ gánh nặng điều phối xuống cấp độ phần cứng nguyên bản của GPU, loại bỏ điểm đồng bộ hóa tai hại. Đặc biệt, kỹ thuật Prompt Lookup Decoding (PLD) chính là "viên đạn bạc" (silver bullet) cho bài toán giới hạn 1ms của TPOT. Kỹ thuật này khai thác sự lặp lại về mặt từ vựng tự nhiên của các phiên hội thoại để kiểm chứng nhiều token trong một lượt xung nhịp mà không làm phình to yêu cầu bộ nhớ hệ thống.

Kết hợp cùng các tinh chỉnh hệ điều hành chuyên sâu như TCMalloc, Thread Pinning, và triệt tiêu log vi mô, cấu trúc máy chủ được tạo ra sẽ không chỉ tuân thủ nghiêm ngặt mọi nguyên tắc Anti-Cheating của bài thi mà còn trở thành một kiến trúc serving có hiệu năng và độ ổn định cao nhất có thể đạt được trên giới hạn toán học của phần cứng hiện tại. Lợi thế kỹ thuật thu được từ việc phá vỡ giới hạn CPU sẽ đưa các chỉ số đo lường ERS tiệm cận về hệ số nhân 1.0 tuyệt đối.

Nguồn trích dẫn

1. LFM2.5-1.2B-Thinking: On-Device Reasoning Under 1GB — Blog - Liquid AI, truy cập vào tháng 7 17, 2026, <https://www.liquid.ai/blog/lfm2-5-1-2b-thinking-on-device-reasoning-under-1gb>

2. LiquidAI/lfm2.5-1.2b-instruct - Ollama, truy cập vào tháng 7 17, 2026, <https://ollama.com/LiquidAI/lfm2.5-1.2b-instruct>
3. Sandwich: Joint Configuration Search and Hot-Switching for Efficient CPU LLM Serving, truy cập vào tháng 7 17, 2026, <https://arxiv.org/html/2507.18454v2>
4. Sandwich: Separating Prefill-Decode Compilation for Efficient CPU LLM Serving - arXiv, truy cập vào tháng 7 17, 2026, <https://arxiv.org/html/2507.18454v1>
5. Can Scheduling Overhead Dominate LLM Inference Performance? A Study of CPU Scheduling Overhead on Two Popular LLM Inference Systems - MLSys @ WukLab, truy cập vào tháng 7 17, 2026, https://mlsys.wuklab.io/posts/scheduling_overhead/
6. Model Runner V2: A Modular and Faster Core for vLLM | vLLM Blog, truy cập vào tháng 7 17, 2026, <https://vllm.ai/blog/2026-03-24-mrv2>
7. Release Notes — TensorRT LLM - GitHub Pages, truy cập vào tháng 7 17, 2026, <https://nvidia.github.io/TensorRT-LLM/release-notes.html>
8. Hybrid Models as First-Class Citizens in vLLM – PyTorch, truy cập vào tháng 7 17, 2026, <https://pytorch.org/blog/hybrid-models-as-first-class-citizens-in-vllm/>
9. Introducing LFM2.5: The Next Generation of On-Device AI — Blog - Liquid AI, truy cập vào tháng 7 17, 2026, <https://www.liquid.ai/blog/introducing-lfm2-5-the-next-generation-of-on-device-ai>
10. Liquid LFM2.5: How To Run & Fine-tune | Unsloth Documentation, truy cập vào tháng 7 17, 2026, <https://unsloth.ai/docs/models/tutorials/lfm2.5>
11. sam860/lfm2.5:1.2b - Ollama, truy cập vào tháng 7 17, 2026, <https://ollama.com/sam860/lfm2.5:1.2b>
12. LFM2.5-1.2B-Instruct - Liquid Docs - Liquid Foundation Models, truy cập vào tháng 7 17, 2026, <https://docs.liquid.ai/lfm/models/lfm25-1.2b-instruct>
13. vLLM Quickstart: High-Performance LLM Serving - in 2026 - Glukhov.org, truy cập vào tháng 7 17, 2026, <https://www.glukhov.org/llm-hosting/vllm/vllm-quickstart/>
14. vLLM in 2026: Architectural Challenges and Performance Optimizations S82059 | GTC San Jose 2026 | NVIDIA On-Demand, truy cập vào tháng 7 17, 2026, <https://www.nvidia.com/en-us/on-demand/session/gtc26-s82059/>
15. Performance degradation due to CPU bottleneck when serving embedding models to GPUs · Issue #11320 · vllm-project/vllm - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/11320>
16. Model Runner V2 Design Document - vLLM, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/latest/design/model_runner_v2/
17. vLLM Model Runner V2 on GPU Cloud: Deploy MRV2 for Faster LLM Inference (2026), truy cập vào tháng 7 17, 2026, <https://www.spheron.network/blog/vllm-model-runner-v2-mrv2-deployment-guide/>
18. [Model Runner V2][Bug]: The _gumbel_sample_kernel exhibits poor, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/40755>
19. vllm.config.vllm, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/stable/api/vllm/config/vllm/>
20. The current vLLM CPU backend is not working properly, truy cập vào tháng 7 17,

2026,

<https://discuss.vllm.ai/t/the-current-vllm-cpu-backend-is-not-working-properly/2708>

21. Efficient LLM Serving - OpenVINO™ documentation, truy cập vào tháng 7 17, 2026, https://docs.openvino.ai/2025/model-server/ovms_docs_llm_reference.html
22. model_server/docs/model_server_rest_api_chat.md at main - GitHub, truy cập vào tháng 7 17, 2026, https://github.com/openvinotoolkit/model_server/blob/main/docs/model_server_rest_api_chat.md
23. Speculative Decoding: Types and Optimizations - Aussie AI, truy cập vào tháng 7 17, 2026, <https://www.aussieai.com/research/speculative-decoding>
24. Prompt Lookup Decoding - Aussie AI, truy cập vào tháng 7 17, 2026, <https://www.aussieai.com/research/prompt-lookup-decoding>
25. OpenAI API completions endpoint - OpenVINO™ documentation, truy cập vào tháng 7 17, 2026, https://docs.openvino.ai/nightly/model-server/ovms_docs_rest_api_completion.html
26. Architecture Overview - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/latest/design/arch_overview/
27. Code Review: Deep Dive into vLLM's Architecture and Implementation Analysis of OpenAI-Compatible Serving (1/2) | Zerohertz, truy cập vào tháng 7 17, 2026, <https://zerohertz.github.io/vllm-openai-1/>
28. Optimization and Tuning - vLLM Ascend, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/projects/ascend/en/latest/developer_guide/performance_and_debug/optimization_and_tuning.html
29. C++ memory allocation mechanism performance comparison (tcmalloc vs. jemalloc), truy cập vào tháng 7 17, 2026, <https://stackoverflow.com/questions/7852731/c-memory-allocation-mechanism-performance-comparison-tcmalloc-vs-jemalloc>
30. LFM2.5-230M: Built to Run Anywhere — Blog - Liquid AI, truy cập vào tháng 7 17, 2026, <https://www.liquid.ai/blog/lfm2-5-230m>
31. Installation with CPU - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.6.6/getting_started/cpu-installation.html
32. Installation with CPU - vLLM! - Read the Docs, truy cập vào tháng 7 17, 2026, https://nm-vllm.readthedocs.io/en/latest/getting_started/cpu-installation.html
33. vllm.config, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/v0.9.2/api/vllm/config.html>
34. Inside vLLM: Anatomy of a High-Throughput LLM Inference System, truy cập vào tháng 7 17, 2026, <https://vllm.ai/blog/2025-09-05-anatomy-of-vllm>
35. KV Cache CPU Offload Guide - vLLM Ascend, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/kv_cache_cpu_offload.html
36. Any options to increase vLLM performance? · Issue #2073 - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/issues/2073>
37. vllm.core.scheduler - vLLM Documentation, truy cập vào tháng 7 17, 2026,

- <https://docs.vllm.ai/en/v0.10.1/api/vllm/core/scheduler.html>
38. vLLM V1: A Major Upgrade to vLLM's Core Architecture | vLLM Blog, truy cập vào tháng 7 17, 2026, <https://vllm.ai/blog/2025-01-27-v1-alpha-release>
 39. How continuous batching enables 23x throughput in LLM inference while reducing p50 latency - Anyscale, truy cập vào tháng 7 17, 2026, <https://www.anyscale.com/blog/continuous-batching-llm-inference>
 40. how to shield a cpu from the linux scheduler (prevent it scheduling threads onto that cpu)?, truy cập vào tháng 7 17, 2026, <https://stackoverflow.com/questions/11111852/how-to-shield-a-cpu-from-the-linux-scheduler-prevent-it-scheduling-threads-onto>
 41. How to set a Java thread's cpu core affinity? - Stack Overflow, truy cập vào tháng 7 17, 2026, <https://stackoverflow.com/questions/13392379/how-to-set-a-java-threads-cpu-core-affinity>
 42. Building LLAMA.CPP with BLAS on Android (Termux): OpenBLAS vs BLIS vs CPU Backend : r/LocalLLM - Reddit, truy cập vào tháng 7 17, 2026, https://www.reddit.com/r/LocalLLM/comments/1orfxa9/building_llamacpp_with_blas_on_android_termux/
 43. Sandwich: Joint Configuration Search and Hot-Switching for Efficient CPU LLM Serving, truy cập vào tháng 7 17, 2026, <https://i.cs.hku.hk/~cwu/papers/jtzhao-dac26.pdf>
 44. VLLM Template - Docs - Chutes, truy cập vào tháng 7 17, 2026, <https://chutes.ai/docs/templates/vllm>
 45. vLLM Documentation, truy cập vào tháng 7 17, 2026, <https://docs.vllm.ai/en/v0.10.1/api/vllm/index.html>
 46. AsyncLLMEngine - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.8.1/api/engine/async_llm_engine.html
 47. OpenAI-Compatible Server - vLLM Documentation, truy cập vào tháng 7 17, 2026, https://docs.vllm.ai/en/v0.8.2/serving/openai_compatible_server.html
 48. Chapter 2. Using custom chat templates with AI Inference and Podman, truy cập vào tháng 7 17, 2026, https://docs.redhat.com/en/documentation/red_hat_ai_inference/3.4/html/extending_red_hat_ai_inference_with_tool_calling_capabilities/tools-using-custom-chat-templates-with-models_tool-calling
 49. froggeric/Qwen-Fixed-Chat-Templates - Hugging Face, truy cập vào tháng 7 17, 2026, <https://huggingface.co/froggeric/Qwen-Fixed-Chat-Templates>
 50. Private LLMs Scaling Made Easy. Large Language Models (LLMs) are widely... | by Thanabordee Noun | Medium, truy cập vào tháng 7 17, 2026, <https://medium.com/@original2547/private-llms-scaling-made-easy-95cdc93e381b>
 51. Chat Template Editor · vllm-project vllm · Discussion #9644 - GitHub, truy cập vào tháng 7 17, 2026, <https://github.com/vllm-project/vllm/discussions/9644>